

--{ python для пентестера }--

Ethical Hacking Tutorial Group The Codeby

представляет



PYTHON

«Для Пентестера»

Programming Language..



-- { ПРОДВИНУТЫЙ УРОВЕНЬ } --

Пишем конвертеры для полезных нагрузок

При работе с полезными нагрузками, очень часто требуется их кодировать или декодировать, если сервер принимает/отправляет данные в закодированном виде.

Рассмотрим наиболее часто встречающиеся варианты:

Кодировка и декодирование Base64

О стандарте base64 <https://ru.wikipedia.org/wiki/Base64>

Для работы с base64 можно использовать одноимённый модуль. Метод `b64encode` принимает байты в качестве аргумента. Для строки требуется указать нужную кодировку.

```
import base64

print(base64.b64encode(bytes('Здесь ваша строка', 'utf-8')))
```

```
b'0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA='

Process finished with exit code 0
```

С использованием переменных этот же код выглядит так:

```
import base64

s = 'Здесь ваша строка'
b = s.encode("UTF-8")
e = base64.b64encode(b)
print(e)
```

```
b'0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA='
```

Избавиться в выводе от префикса `b` можно с помощью перевода байтов в строку с кодированием в `utf-8`.

```
import base64

enc_bytes = base64.b64encode("Здесь ваша строка".encode("utf-8"))
enc_str = str(enc_bytes, "utf-8")
print(enc_str)
```

base_tree x

0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA=

При работе со строками на латинице, без указания кодировки, можно делать конвертацию следующим образом:

```
import base64

print(base64.b64encode(b'Here is your line'))
```

base x

b'SGVyZSBpcyB5b3VyIGxpbmU='

Process finished with exit code 0

Префикс `b` обозначает что строка в кавычках является байтовым типом данных.

```
import base64

print(type(base64.b64encode(b'Here is your line')))
```

base x

<class 'bytes'>

Метод `b64decode` соответственно декодирует. Обратите внимание, что декодирование кириллицы не прошло, и мы получили не оригинальный текст, а 8-битную строку.

```
import base64

print(base64.b64decode(bytes('0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA=', 'utf-8')))
print(base64.b64decode(b'SGVyZSBpcyB5b3VyIGxpbmU='))
```

base_two x

b'\xd0\x97\xd0\xb4\xd0\xb5\xd1\x81\xd1\x8c \xd0\xb2\xd0\xb0\xd1\x88\xd0\xb0 \xd1\x81\xcd
b'Here is your line'

Чтобы получить исходный текст, мы должны дополнительно декодировать полученный ответ в Unicode.

Получение исходного текста в обоих вариантах:

```
import base64

x = base64.b64decode(bytes('0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA=', 'utf-8'))
print(x.decode('utf-8'))
print((base64.b64decode(b'0JfQtNC10YHRjCDQstCw0YjQsCDRgdGC0YDQvtC60LA=')).decode('utf-8'))
```

base_two x

Здесь ваша строка

Здесь ваша строка

Кодировка и декодирование urlencode

Кодирование URL

URL — это адрес веб-страницы, например: <http://www.site.com>

HTML-формы по умолчанию используют **application/x-www-form-urlencoded** тип контента для отправки параметров.

URL-адрес ресурса отправляется в интернете в ASCII-кодировке. Если в URL содержатся символы, не входящие в ASCII-кодировку, URL конвертируется. Перекодироваться должны буквы кириллицы, иероглифы. Точно так же строки запроса, содержащие небезопасные и любые не-ASCII символы, также кодируются. Вы должны расшифровать их перед использованием. Или соответственно наоборот закодировать, если вы отправляете запрос на сервер.

Кодировщик URL заменяет небезопасные символы на символы ASCII знаком (%), за которым следуют два шестнадцатирочных числа, которые соответствуют значениям символов из кодировки ISO-8859-1. URL не должен содержать пробелы. Кодировщик URL обычно заменяет пробелы знаком (%20).

Примеры кодирования с помощью встроенной в Python библиотеки **urllib** и метода **quote**

```
import urllib.parse

query = 'Hello World!'
print(urllib.parse.quote(query))
```

url x

Hello%20World%21

```
import urllib.parse

query = 'http://www.bla-bla.com/urlencode()'
print(urllib.parse.quote(query))
```

url x

<http%3A//www.bla-bla.com/urlencode%28%29>

Process finished with exit code 0

Небезопасные символы — это пробел, кавычки, знак меньше, знак больше, знак диез, знак проценты, фигурные скобки, прямой слэш, обратный слэш, тильда, квадратные скобки, гравис. Эти символы также должны всегда быть закодированы.

Десятичное число	Шестнадцатеричное значение	Символ	Кодирование URL
32	20	Пробел	%20
34	22	«	%22
60	3c	<	%3c
62	3e	>	%3e
35	23	#	%23
37	25	%	%25
123	7b	{	%7b
125	7d	}	%7d
124	7c		%7c
92	5c	\	%5c
94	5e	^	%5e
126	7e	~	%7e
91	5b	[	%5b
93	5d	]	%5d
96	60	`	%60

Из-за разных стандартов иногда кодирование пробелов выполняется знаками плюс. Сделать на пайтоне это можно с помощью метода `quote_plus`

```
import urllib.parse

query = 'А и Б'
print(urllib.parse.quote_plus(query))
```

url ×

%D0%90+%D0%B8+%D0%91

Для декодирования применим метод `unquote_plus`

```
import urllib.parse

query = '%D0%90+%D0%B8+%D0%91'
print(urllib.parse.unquote_plus(query))
```

url ×

А и Б

Для всех остальных случаев при декодировании применяется метод `unquote`

```
import urllib.parse

query = 'http%3A//www.bla-bla.com/urlencode%28%29'
print(urllib.parse.unquote(query))
```

url ×

[http://www.bla-bla.com/urlencode\(\)](http://www.bla-bla.com/urlencode())

Кириллица

```
import urllib.parse

query = 'Пример'
print(urllib.parse.quote(query))
query2 = '%D0%9F%D1%80%D0%B8%D0%BC%D0%B5%D1%80'
print(urllib.parse.unquote(query2))
```

url ×

%D0%9F%D1%80%D0%B8%D0%BC%D0%B5%D1%80
Пример

Однако если в строке будет обратный слэш, то мы получим ошибку. Избежать эту проблему можно с помощью "r", который ставится перед началом строки. Или просто экранируя обратным слэшем.

```
import urllib.parse

query = 'C:\Users\Vasya'
print(urllib.parse.quote(query))
```

url ×

File `"/root/PycharmProjects/education/pro_9/url.py"`, line 3
`query = 'C:\Users\Vasya'`
`^`
SyntaxError: (unicode error) 'unicodeescape' codec can't decode

```
import urllib.parse

query1 = 'C:\\Users\\Vasya'
query2 = r'C:\Users\Vasya'
print(urllib.parse.quote(query1))
print(urllib.parse.quote(query2))
```

url ×

C%3A%5CUsers%5CVasya
C%3A%5CUsers%5CVasya

Кодировка и декодирование ASCII

[ASCII](#) (American standard code for information interchange) — название таблицы (кодировки, набора), в которой некоторым распространённым печатным и непечатным символам сопоставлены числовые коды.

С помощью представления ASCII мы можем различить с виду одинаковые символы, но относящиеся к разным языкам. Например AB и AB выглядят одинаково, но посмотрим как отличит их программа.

```
text = 'AB AB'
print(ascii(text))
print('\u0410\u0412 AB')
```

url2 ×

'\u0410\u0412 AB'
AB AB

Функция `ascii()` возвращает строку, содержащую печатаемое представление объекта. Она заменяет все символы не-ASCII в строке, используя для этого экранирование `\x`, `\u` или `\U`.

Код символа можно определить при помощи функции `ord()`. Эта функция получает на вход строку, которая должна состоять ровно из одного символа. Функция возвращает код этого символа. Например, `ord('a')` вернет число 97.

Обратная функция получения по числовому коду его номера называется `chr()`.

```
print(ord('a'))
print(chr(97))
```

url2 x

```
97
a
```

Очень часто кодирование применяется в полезных нагрузках. Например для тестирования на наличие уязвимости XSS. Но так как функции `ord` и `chr` принимают только 1 символ, мы можем кодировать/декодировать значения в цикле.

```
x = 97, 108, 101, 114, 116, 40, 49, 41
for i in x:
    print(chr(i), end='')
# <script>eval(String.fromCharCode (97, 108, 101, 114, 116, 40, 49, 41));</script>
```

url2 x

```
alert(1)
Process finished with exit code 0
```

```
x = list('alert(1)')
for i in x:
    print(ord(i), end=', ')
```

url2 x

```
97, 108, 101, 114, 116, 40, 49, 41,
Process finished with exit code 0
```

Кодировка и декодирование hex

HEX - (hexadecimal) обозначение [шестнадцатеричной системы счисления](#). Довольно популярный формат, используемый во множестве всевозможных полезных нагрузок, например SQL-инъекциях.

Формат hex представляет целое число в шестнадцатеричной форме, имеет вид – 0xAF. Перед числом стоит 0x, дальше следует само число в двоичной форме AF.

```
print("The hexadecimal form of the"
      " ascii value is 'a' is " + hex(ord('a')))
```

hex x

```
The hexadecimal form of the ascii value is 'a' is 0x61
```

Функция `hex()` может принимать много символов класса `int`.

```
print(hex(1234567890))
```

hex ×

0x499602d2

Функция `ascii` также умеет преобразовывать формат `hex`, возвращая строку.

```
print(ascii(0x499602d2))
```

hex ×

1234567890

В сети используются различные представления `hex`, в том числе и виде массива байтов, а также разделённые обратным слешем без нуля в начале.

Примеры разных реализаций кода.

```
import codecs
```

```
print(codecs.encode(b'bla-bla', 'hex'))
```

```
print(codecs.decode('626c612d626c61', 'hex'))
```

```
text = 'bla-bla'
```

```
output = '\\'.join(hex(ord(c))[1:] for c in text)
```

```
print('\\' + output)
```

```
hex_string = "626c612d626c61"
```

```
print(bytes.fromhex(hex_string))
```

```
print(bytearray.fromhex(hex_string))
```

hex ×

b'626c612d626c61'

b'bla-bla'

\\x62\\x6c\\x61\\x2d\\x62\\x6c\\x61

b'bla-bla'

bytearray(b'bla-bla')